

BinaryOperator<T> [If two inputs are same type and return type is also same]

It is a predefined functional interface available in java.util.function sub package.

It is a functional interface in Java that represents an operation upon two operands of the SAME TYPE, producing a result of the SAME type as the operands. [All three(two inputs and one result) must be of same type]

This is a specialization of BiFunction<T,T,T> for the case where the operands and the result are all of the same type.

It has two parameters of same type, T, which represents both the operand types and the result type.

```
@FunctionalInterface
public interface BinaryOperator<T> extends BiFunction<T,T,T>
{
    public abstract T apply(T x, T y);
}

package com.ravi.unary_and_binary;

import java.util.function.BinaryOperator;

public class BinaryOperatorDemo1
{
    public static void main(String[] args)
    {
        BinaryOperator<Double> addAmount = (amt1, amt2) -> amt1 + amt2;
        Double total = addAmount.apply(5000.0, 3000.0);
        IO.println("Total Amount: " + total);

        BinaryOperator<String> concat = (str1, str2) -> str1 + str2;
        IO.println(concat.apply("Java", "Technology"));

        BinaryOperator<Integer> getSub = (n1, n2) -> n1-n2;
        Integer sub = getSub.apply(200, 100);
        IO.println("Sub is :"+sub);
    }
}
```

Complete Details of all the Functional Interfaces :

- | | |
|----------------------|-------------------------|
| 1) Predicate<T> | boolean test(T x); |
| 2) Consumer<T> | void accept(T x); |
| 3) Function<T,R> | R apply(T x); |
| 4) Supplier<T> | T get(); |
| 5) BiPredicate<T,U> | boolean test(T x, U y); |
| 6) BiConsumer<T, U> | void accept(T x, U y); |
| 7) BiFunction<T,U,R> | R apply(T x, U y); |
| 8) UnaryOperator<T> | T apply(T x) |
| 9) BinaryOperator<T> | T apply(T x, T y) |

Does an interface extends a class ?

No, an interface can't extend a class, An interface can extend another interface.

In order to support upcasting concept in java, all the public (not protected) and non final methods of Object class are re-declared inside an interface as an abstract method by java compiler if that interface does not extend from another interface.

```
package com.ravi.interface_members;

interface Callable
{
    /*public and non final method of Object class is
    automatically added by java compiler as an
    abstract method to support up-casting
    */
    public String toString();
    public int hashCode();
    public boolean equals(Object obj);
}

class Call extends Object implements Callable
{
    @Override
    public String toString()
    {
        return "Java";
    }
}

public class InterfaceMember1
{
    public static void main(String[] args)
    {
        Callable c = new Call();
        //c.toString();

        IO.println(c.toString());
        IO.println(c.hashCode());
        c.equals(null);
    }
}

package com.ravi.interface_members;
```

@FunctionalInterface
interface Flyable
{
 void fly();
 public String toString();
 public int hashCode();
 public boolean equals(Object obj);
}

Valid Functional interface, If we will not write hashCode(), equals() and toString() then automatically java compiler will add as an abstract method

```
public class InterfaceMemberDemo2
{
    public static void main(String[] args) {
        // TODO Auto-generated method stub

    }
}
```

Interface from Java 9V

We can write private static and private non static (non public) methods inside an interface from java 9 version.

The main purpose of providing these two methods inside an interface are as follows :

1) Code Reusability
If two or more than two default/public static methods want to share a common code (Helper Method code) then we can write these common code in private methods so it will enhance code reusability.

2) Hide the Logic from Outer World
By writing the logic in private static and private non static methods the actual logic is not visible to the outer world due to private method.

Note : Till JDK 1.7, Interface was fully abstract but from JDK 1.8V onwards we can write method body inside an interface so It is not fully abstract. Fully abstraction we can achieve without method body only.

```
package com.ravi.interface_members;

interface Moveable
{
    int SPEED = 100;    //JDK 1.0

    void m1();          //JDK 1.0

    default void m2()    //JDK 1.8
    {
        m6(); //calling private non static method
    }

    default void m3()    //JDK 1.8
    {
        m6(); //calling private non static method
    }

    static void m4()     //JDK 1.8 [public static ]
    {
        m5(); //calling private static method
    }

    private static void m5() //JDK 9 [private static method]
    {
        //Helper method [used to write some common code]
        IO.println("private static method of interface");
    }

    private void m6()    //JDK 9 [private non static method]
    {
        //Helper method [used to write some common code]
        IO.println("private non static method of interface");
    }
}

class Test implements Moveable
{
    @Override
    public void m1()
    {
        IO.println("M1 method is overridden");
    }
}

public class InterfaceNewFeature
{
    public static void main(String[] args)
    {
        Moveable m = new Test();
        m.m1();
        m.m2();
        m.m3();
        Moveable.m4();
    }
}
```

What is Marker Interface : [We will learn after exception & Multithreading]

If an interface does not contain any field or methods, Basically an empty interface is known as Marker interface OR tag interface.

Example :
interface Drawable //Marker interface
{
 //Empty interface
}

In java, we have few predefined marker interfaces are available which are as follows :

- 1) java.lang.Cloneable
- 2) java.io.Serializable
- 3) java.util.RandomAccess

Marker interface is used to provide additional information to JVM regarding the object like Object is Cloneable OR Serializable OR Randomly Accessible OR not.

Internally JVM will verify using instanceof operator.
Example :
final class Product implements Cloneable
{
}

```
Product p1 = new Product();

if(p1 instanceof Cloneable)
{
    //create duplicate object using clone();
}
else
{
    //throwing an exception
}
```

****What are the differences between abstract class and interface ?

The following are the differences between abstract class and interface.

- 1) An abstract class can contain instance variables but interface variables are by default public , static and final (no instance variable).
- 2) An abstract class can have state (Object properties) of an object but interface can't have state of an object.
- 3) An abstract class can contain constructor but inside an interface we can't define constructor.
- 4) An abstract class can contain instance and static blocks but inside an interface we can't define any blocks.
- 5) Abstract class cannot be the target for Lambda expression but the target of Lambda expression must be a functional interface.
- 6) By using abstract class multiple inheritance is not possible but by using interface we can achieve multiple inheritance.
- 7) Abstract classes are meant for providing extending features but interfaces are used to provide Loose coupling [LCRP -> Loose coupling Runtime Poly]

-----OOPs Completed-----